

Maestro / MPTK 集成（虚拟设备与适配器）

本页面介绍了可选的 **Maestro / MidiPlayerTK (MPTK)** 集成层。

它提供了两种高级工作流：

1. 将 **MPTK 作为虚拟 MIDI 输出设备 (Sink)**

将 `MidiManager` 的事件路由到 MPTK 合成器中（实时播放）。

2. 将 **MPTK 作为虚拟 MIDI 输入设备 (Source)**

获取 MPTK 的回调（来自 `MidiFilePlayer` 或 `MidiStreamPlayer`），并将其作为等效的 MIDI 事件注入到 `MidiManager` 中，就像它们来自真实设备一样。

术语说明：在本插件中，“虚拟设备”是指在 `MidiManager` 的设备列表和事件管线中参与运行的软件终端，它不涉及平台的原生 MIDI 后端。

启用 / 禁用（编译时）

该集成由脚本定义符号控制：

- `FEATURE_USE_MPTK`
- `MPTK_PRO`（Maestro Pro 功能：`OnMidiEvent` Pipeline、Writer、InnerLoop、ListPlayer、Spatializer 等）

当该定义不存在（或未安装 MPTK）时，集成脚本会编译为空桩 (Stubs)，从而确保项目仍能正常构建。Pro 专用 API 会额外用 `#if MPTK_PRO` 进行守卫。

源码位于 `Assets/MIDI/Scripts/Integrations/MPTK/`（与 Chunity 一样放在 `Integrations/` 下）。

如何启用

在 Unity 中：

- **Project Settings** → **Player** → **Other Settings** → **Script Compilation** → **Scripting Define Symbols**
- 添加：`FEATURE_USE_MPTK`
- 使用 Maestro Pro 时再添加：`MPTK_PRO`

兼容的 Maestro 版本

集成测试基准为 **Maestro / MidiPlayerTK 2.21.x**（Free 或 Pro）。Free 路径在 2.15–2.20 上也可能可用；封装 2.17+ API 的 Pro 助手（语音暂停/恢复、Orientation、和弦进行、实时 SoundFont 效果）以 2.21 系 Pro 为准。

运行时助手（组件 / MCP）

助手	作用	Define / MCP
MptkSynthEffectsController	运行时 SoundFont filter / reverb / chorus	MPTK_PRO · midi-mptk-effects-configure
MptkVoiceLifecycle / MptkMidiDevice.PauseVoices	平滑语音静音 / 恢复	MPTK_PRO · midi-mptk-voice-lifecycle
MptkDistanceAudioSettings	距离衰减 + Pro Orientation	Free 距离 · Pro Orientation · midi-mptk-distance-audio
MptkChordProgressionPlayer	播放 Maestro 进行预设	MPTK_PRO · midi-mptk-chord-progression
全局旋钮（MptkUtility / MptkBootstrap）	MPTK_RunInBackground / MPTK_AudioListener（可在 EnsureReady 时可选应用）	FEATURE_USE_MPTK · midi-mptk-global-settings
实验性 Velocity 曲线	MPTK_VelocityAttenuation（经 MptkUtility，慎用）	FEATURE_USE_MPTK · midi-mptk-velocity-attenuation
Visual Scripting / Timeline	Effects / Voice / Distance / Chord / Global 单元与流控制标记	FEATURE_USE_VISUALSCRIPTING / FEATURE_USE_TIMELINE

诊断：midi://mptk/setup-report。鼓组预设：MptkUtility.BuildDrumPresetsText()、midi://mptk/soundfont 或 midi-mptk-bank-program 的 action=list-drums。

效果与 Orientation 可能增加 CPU 或降低体感音量；默认保持关闭，需要时再开并调整 Global Volume。

包含的内容

虚拟设备注册与输入注入

`MidiManager` 为软件终端提供了辅助 API：

- 注册/注销虚拟设备 ID（作为输入、输出或两者兼有）
- 直接向内部管线注入 MIDI 事件（例如“模拟此设备发送了一个 Note On”等）

当您想要模拟设备、桥接其他系统或对处理器进行单元测试时，可以使用这些 API。

MPTK 虚拟 MIDI 设备 (Sink)

`MptkMidiDevice` 实现了 MIDI 事件处理器接口，并将事件转发到内部的 MPTK `MidiStreamPlayer`。

在以下场景中使用它：

- `MidiManager` → MPTK 合成器播放
- 创建一个与真实设备并列显示的“软件输出 deviceId”

MPTK → MidiManager 适配器 (Source)

`MptkMidiManagerInputAdapter` 可以订阅 MPTK 回调事件（通常由 MPTK 播放器暴露），并将匹配的 MIDI 1.0 风格事件注入到 `MidiManager` 中。

在以下场景中使用它：

- 使用 MPTK 播放/实时生成的内容来驱动您现有的 `MidiManager` 处理器
- 让 MPTK 表现得像一个虚拟的输入设备

便捷工具

`MptkUtility` 封装了常用的设置逻辑：

- 确保 MPTK 全局变量存在
- 创建 MPTK 播放器
- 创建并注册虚拟设备
- 创建适配器
- 提供了将 MIDI 事件作为虚拟输入注入的辅助方法
- 设置校验（`ValidateSetup`）

合成前 rewrite（`MptkMidiEventPipeline`，**Maestro Pro**）

通过 `OnMidiEvent` 进行 keep / drop / inject。详情参见后文 Bootstrap / 诊断章节中的“`MptkMidiEventPipeline`”。

Writer / 外部 URI（`MptkWriterBridge`、`MptkExternalPlayback`，**Maestro Pro**）

`MPTKWriter` / `MidiExternalPlayer` 的薄封装。详情参见后文“`MptkWriterBridge` / `MptkExternalPlayback`”。

内侧循环（`MptkInnerLoopController`，**Maestro Pro**）

`MPTK_InnerLoop` 的薄封装。详情参见后文 Clock 同步章节中的“循环区间”。

Bootstrap / SmfPlayer / 诊断

简化设置、连接 SMF 播放，并提供 Panic / Bank Select 与调试辅助。

MptkBootstrap — 一键启动

MptkBootstrap 会自动执行以下操作：

- 确保存在 `MidiPlayerGlobal`
- 注册虚拟设备 ID（输入/输出）
- 创建 `MptkMidiDevice` 并注册为 `MidiManager` 处理器

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

public sealed class MyMptkSetup : MonoBehaviour
{
    void Start()
    {
        var bootstrap = gameObject.AddComponent<MptkBootstrap>();
        bootstrap.EnsureReady();
    }
}
#endif
```

推荐的默认 ID：`MptkUtility.DefaultOutputDeviceId`（`mptk:internal`）

MptkSmfPlayerOutput — SmfPlayer 集成

自动将 `SmfPlayer` 的 `outputDeviceId` 设置为 MPTK 虚拟设备。

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi;
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

[RequireComponent(typeof(SmfPlayer))]
public sealed class MySmfWithMptk : MonoBehaviour
{
    void Awake()
    {
        var output = gameObject.AddComponent<MptkSmfPlayerOutput>();
        output.Configure(); // SmfPlayer.outputDeviceId = "mptk:internal"
    }
}
#endif
```

若在同一 `GameObject` 上放置 `MptkBootstrap`，`MptkSmfPlayerOutput` 会复用它。

MptkDspUmpSequenceOutput — Scriptable Audio UMP 集成

当 `FEATURE_USE_MPTK` 与 `FEATURE_SCRIPTABLE_AUDIO` 均启用时，可将经由 `MidiDspUmpMidi2OutBridge` 的 DSP 已调度 UMP 路由到 MPTK 虚拟输出。

源码位于 `Assets/MIDI/Scripts/Integrations/MPTK/ScriptableAudio/`（程序集 `jp.kshoji.midi.mptk.scriptableaudio`），布局与 Chunity / Timeline 的可选子文件夹一致。

1. 将 `MptkDspUmpSequenceOutput` 添加到与 UMP Bootstrap / Scheduler 相同的 GameObject（或其接线目标）
2. 按需将 `MptkBootstrap` 一并放置（缺失时提供自动创建选项）
3. 在 Play 模式下，`Configure()`（或 `autoConfigureOnAwake`）会将 UMP Out 的输出目标设置为 `mptk:internal` 等

如果不希望与内置参考合成器重复发声，请启用组件的 `muteBuiltInSynthWhenActive`。详情参见 [UMP 序列播放（DSP 同步）](#)。

Panic / All Notes Off / Reset

`MptkMidiDevice.PanicAll()` 会执行以下操作：

- 使用 `MPTK_StopEvent` 停止正在追踪的 Note On
- 向所有通道发送 CC 120 (All Sound Off) / CC 123 (All Notes Off)

当接收到 CC 120 / 121 / 123 时，也会按通道执行等效处理。在 MIDI Reset（`OnMidiReset`）时会调用 `PanicAll()`。

也可以通过 `MptkBootstrap.Panic()` 执行相同操作。请将目标 `deviceId` 设置为 `mptk:internal` 等。

Bank Select + Program Change

`MptkMidiDevice` / `MptkMidi2Device` 会按通道追踪 CC 0 (Bank MSB) / CC 32 (Bank LSB)，并在 Program Change 时向 MPTK 发送 `MPTKController.BankSelectMsb` + `MPTKCommand.PatchChange`。即使使用非 General MIDI 的 SoundFont，音色切换也能保持稳定。

设置校验（`MptkUtility.ValidateSetup`）

可在 Play 模式下检查 MPTK 集成的状态。

```
var report = MptkUtility.ValidateSetup(bootstrap.Sink, bootstrap.DeviceId);
Debug.Log(report.ToSummary());
```

检查项	严重程度
<code>MidiPlayerGlobal</code> 存在	Error
<code>MidiManager</code> 可用	Error
虚拟设备已注册	Error
<code>MidiStreamPlayer</code> / <code>AudioSource</code>	Error / Warning
SoundFont 已加载	Error
已应用的 SF 名称（指定 <code>expectedSoundFontName</code> 时）	Warning

启用 `autoValidateOnStart` 时， `MptkBootstrap` 会在启动时自动校验。

调试： `MptkEventTap`

一个可选组件，将桥接上的 MIDI 事件记录到 Console。

属性	说明
<code>direction</code>	<code>ToMptk</code> （Sink 方向） / <code>FromMptk</code> （适配器注入方向）
<code>filterDeviceId</code>	仅记录特定 deviceId
<code>logNoteEvents</code> / <code>logControlChange</code>	事件类型过滤

在场景中添加一个并设置 `isLoggingEnabled = true`，即可启用来自 `MptkMidiDevice` / `MptkMidiManagerInputAdapter` 的日志。

`MptkMidiEventPipeline` — 合成前 rewrite（Maestro Pro）

挂钩 `MidiFilePlayer` / `MidiExternalPlayer` 的 `OnMidiEvent`，无需编辑 SMF 即可 keep / drop / inject。

项目	内容
定义	<code>FEATURE_USE_MPTK + ** MPTK_PRO **</code> （ <code>OnMidiEvent</code> / <code>PlayDirect</code> 为 Pro）
组件	<code>MptkMidiEventPipeline</code>
Mapping SO	<code>Create > MIDI > MPTK > Event Mapping</code> （ <code>MptkMidiEventMapping</code> ）
样例	<code>Assets/MIDI/Samples/MPTK/Scripts/MptkMidiEventPipelineSample.cs</code>

内置开关示例：琶音 inject、丢弃 `PatchChange`、`SetTempo` 随机化。代码侧为 `Filter`（非主线程——禁止 Unity API）。Mapping 的 `UnityEvent` 会排队到主线程。

```
#if FEATURE_USE_MPTK && MPTK_PRO
using jp.kshoji.unity.midi.mptk;
using MidiPlayerTK;
using UnityEngine;

public sealed class MyPipelineSetup : MonoBehaviour
{
    public MidiFilePlayer filePlayer;

    void Start()
    {
        var pipeline = gameObject.AddComponent<MptkMidiEventPipeline>();
        pipeline.Source = filePlayer;
        pipeline.EnableArpeggio = true;
        pipeline.Filter = e =>
            e.Command == MPTKCommand.NoteOn && e.Channel == 9
                ? MptkMidiEventPipeline.Result.Drop
                : MptkMidiEventPipeline.Result.Keep;
    }
}
#endif
```

与 InputAdapter 搭配使用时，请仅将 rewrite 之后的事件（OnEventNotesMidi）送往虚拟输入，以避免重复发声。

MptkWriterBridge / MptkExternalPlayback — Writer / 外部 URI（Maestro Pro）

项目	内容
定义	FEATURE_USE_MPTK + ** MPTK_PRO **（MPTKWriter / MidiExternalPlayer）
Writer	MptkWriterBridge — MidiSequenceAsset / SMF 字节 / MidiDB / ImportFromPlayer → Write / 内存 Play
External	MptkExternalPlayback — file:// / http(s):// 播放，可选连接 InputAdapter（ConnectInputAdapterOnPlay）
样例	Assets/MIDI/Samples/MPTK/Scripts/MptkWriterExternalSample.cs（生成 / Join / 临时 .mid / URI）

```
#if FEATURE_USE_MPTK && MPTK_PRO
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

public sealed class MyWriterExternalSetup : MonoBehaviour
{
    public MidiSequenceAsset sequence;

    void Start()
    {
        var writer = gameObject.AddComponent<MptkWriterBridge>();
        var external = gameObject.AddComponent<MptkExternalPlayback>();
        writer.LoadFromSequenceAsset(sequence);
        var path = writer.WriteToTempFile();
        external.ConnectInputAdapterOnPlay = true;
        external.PlayFile(path);
    }
}
#endif
```

播放列表 / SoundFont / 延迟派发 / 通道控制

组件	依据 Demo	Pro 守卫	样例
MptkListPlayerBridge	TestMidiListPlayer	MidiListPlayer → MPTK_PRO	MptkListPlayerSample.cs
MptkSoundFontLoader	TestLoadSF	运行时 Load → MPTK_PRO	MptkPhaseDUilitiesSample.cs
MptkDelayedNoteDispatcher	CatchMusic	无（OnEventNotesMidi）	同上
MptkFilePlayerChannels	MidiChannel*	PlayDirect / StopDirect → MPTK_PRO	同上

List： SetPlaylist / AddMidi / PlayAtIndex / OverlayTimeMs 。可在曲目开始时通过 ConnectInputAdapterOnSongStart 连接到 InputAdapter。

SoundFont： Load URL / file:// / StreamingAssets / 内置名称。通过 MptkUtility.ValidateSetup(..., expectedSoundFontName: "...") 校验已应用的名称。

Delayed： 将已 mute 的 FilePlayer 的 OnEventNotesMidi 按 ms 或 tick 偏移排队，并送往 Stream 或 MidiManager 。不附带可视化演示，仅提供契约。

Channels： SetChannelEnabled / SetSoloChannel / SetDrumsOnly / SetSustain / PlayDirect / StopDirect / Panic 。

Spatializer / Visual Scripting / Timeline

组件	依据 Demo	Pro / 宏	样例
MptkSpatializerHost	SimplestMidiSpatializer	MidiSpatializer → MPTK_PRO	MptkSpatializerSample.cs
MptkSpatializerLayout	—	deviceId / 位置 SO	—
MptkDistanceAudioSettings	—	DistanceAttenuation + Orientation (Pro)	MCP midi-mptk-distance-audio
VS 节点 (Pipeline 等)	P2	FEATURE_USE_VISUALSCRIPTING	通过 Window 菜单注册
MptkFilePlayerMarker + Receiver	Seek / InnerLoop	FEATURE_USE_TIMELINE	MptkTimelineMarkerExample.cs

请将 Maestro Pro 的 **MidiSpatializer** prefab 指定给 Host。可听的 3D 还需 Unity 空间化插件。各 synth 的音符以 deviceIdPrefix:index (或 Layout 后缀) 注入 MidiManager。

Layout SO：Create > MIDI > MPTK > Spatializer Layout (MptkSpatializerLayout)。

距离衰减与 Spatializer 的区别

这些是 不同功能。

功能	组件 / API	作用
Spatializer (Track/Channel)	MptkSpatializerHost + Maestro MidiSpatializer	一份 MIDI → 多个 synth (按 track 或 channel) 挂到 3D 锚点
距离衰减	MptkDistanceAudioSettings / MPTK_DistanceAttenuation	单个 synth 音量随与听者距离变化 (min/max、超距 pause)
Orientation (Pro)	同上 / MPTK_Orientation	相对 MPTK_AudioListener 的角度声像与前滤波

多乐器布局用 Spatializer。Stream/File (或各 spatial synth) 要对距离/角度响应时, 用 MptkDistanceAudioSettings (或 Host 的 **Arrange 时 Distance / Orientation**)。MCP：midi-mptk-distance-audio。

和弦进行生成与和弦识别的区别

功能	API	作用
进行生成 (MPTK Pro)	MptkChordProgressionPlayer / midi-mptk-chord-progression	在 MidiStreamPlayer 上播放 Maestro 情绪进行预设
和弦识别 (套件)	ChordRecognition / midi-chord-state	根据当前按下的输入音符推断和弦名

生成负责发音, 识别负责解析输入, 二者不可互换。

鼓组预设一览：midi://mptk/soundfont, 以及 midi-mptk-bank-program 的 action=list-drums。

MptkIntegrationSampleScene 已添加 Pipeline / Writer / InnerLoop 的可选 GUI 开关 (以及 Effects / Voice / Distance / Chord)。

示例场景 / 输出预设 / 编辑器试听

专用示例场景

项目	路径
场景	Assets/MIDI/Samples/MPTK/Scenes/MptkIntegrationSampleScene.unity
脚本	Assets/MIDI/Samples/MPTK/Scripts/MptkIntegrationSampleScene.cs

启用 `FEATURE_USE_MPTK` 时，可从 GUI 尝试以下操作：

- `MptkBootstrap` 设置 / 校验 / Panic
- 通过 `MidiOutputRoutingPreset` 发送 Note
- 使用 `SmfPlayer` + `MptkSmfPlayerOutput` 试听 SMF
- Pipeline (arp) / Writer (demo 音符或 `SequenceAsset`) / InnerLoop 区间 / Effects / Voice / Distance / Chord

请在 Inspector 中将 `Assets/MIDI/Scripts/Integrations/MPTK/Presets/MptkVirtualOutput.preset.asset` 指定给 `outputPreset`。

套件通用 `outputDeviceId` 预设

借助 `MidiOutputRoutingPreset`（Assets > Create > MIDI > Output Routing Preset）与 `MidiOutputRouting` 辅助工具，可在各组件之间共享 MPTK 输出目标。

可用于具有 `outputDeviceId` / `outputPreset` 字段的发送组件。

解析顺序：**`outputDeviceId`（显式）> `outputPreset` > 第一个输出设备**

随附预设：

- `Assets/MIDI/Scripts/Integrations/MPTK/Presets/MptkVirtualOutput.preset.asset` — 路由到 `mptk:internal`

```
using jp.kshoji.unity.midi.foundation;

// 套件发送示例
MidiOutputRouting.CreateBuilder(outputDeviceId, outputPreset, group)
    .Channel(channel)
    .NoteOn(60, 100);
```

编辑器 SMF 试听（Window > MIDI > SMF Preview）

已在 SMF Preview 窗口中添加 **Preview Audio (MPTK)** 按钮（需要 `FEATURE_USE_MPTK`）。

1. 加载 `.mid`
2. 点击 **Preview Audio (MPTK)** → 进入 Play Mode, `SmfPlayer` 通过 MPTK 虚拟输出播放
3. 点击 **Stop Preview** 停止

内部组件：

组件	作用
SmfPreviewPlaybackRequest	从编辑器 → Play Mode 的播放请求
SmfPreviewPlaybackHost	在 Play Mode 中启动 SmfPlayer
MptkSmfPreviewPlaybackHook	预先初始化 MPTK Bootstrap

Clock 同步 / 循环区间 / MIDI 2.0 输入 / MPE / Visual Scripting

Clock 同步 — MptkClockSyncBridge

将外部 `MidiClockSync` 与 MPTK 播放（`SmfPlayer` / `MidiFilePlayer`）联动。

模式	行为
Follow	将估算的 BPM 反映到 <code>SmfPlayer.tempoBpm</code> / <code>MidiFilePlayer.MPTK_Tempo</code>
Step	每个外部 Clock 拍将 <code>SmfPlayer</code> 推进一拍
Free	忽略外部 Clock

启用 `syncTransportToClock` 后，会配合外部 Start / Stop 开始/停止 SMF / MPTK 文件播放。

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi;
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

public sealed class MyClockBridge : MonoBehaviour
{
    public MidiClockSync clockSync;
    public SmfPlayer smfPlayer;

    void Awake()
    {
        gameObject.AddComponent<MptkBootstrap>().EnsureReady();
        gameObject.AddComponent<MptkSmfPlayerOutput>().Configure();
        gameObject.AddComponent<MptkClockSyncBridge>();
    }
}
#endif
```

若要从 MPTK 回调将 Clock 事件注入 `MidiManager`，现有的 `MptkMidiManagerInputAdapter` 会转发 Timing Clock / Start / Stop / Continue。

循环区间 — MptkInnerLoopController（Maestro Pro）

`MptkClockSyncBridge` 负责外部 **Clock** 的速度/传输同步。而 `MptkInnerLoopController` 则封装 **MPTK FilePlayer / ExternalPlayer** 播放中的**内侧循环**（`MPTK_InnerLoop`）。它与 `SmfPlayer` 或 `DSP Scheduler` 的 loop 是不同的引擎。

项目	内容
定义	<code>FEATURE_USE_MPTK</code> + <code>** MPTK_PRO **</code> (<code>MPTK_InnerLoop</code>)
组件	<code>MptkInnerLoopController</code> — <code>Start</code> / <code>Resume</code> / <code>End</code> / <code>Max</code> / <code>Finished</code>
事件	<code>OnLoopStart</code> / <code>OnLoopResume</code> / <code>OnLoopExit</code> (主线程)。 <code>PhaseFilter</code> 在 MIDI 线程 (禁止 Unity API)
辅助	<code>MeasureToTick</code> / <code>SetLoopByMeasure</code> (拍号→tick)
样例	<code>Assets/MIDI/Samples/MPTK/Scripts/MptkInnerLoopSample.cs</code>

由于 `MPTK_InnerLoop` 会在 MIDI 加载时被清除，默认会在 `OnEventStartPlayMidi` 时重新应用参数 (`ReapplyOnStartPlay`) 。

Free 回退： 无 Pro 时可用 `MPTK_MidiLoaded.MPTK_TickStart` / `MPTK_TickEnd` + `MPTK_MidiAutoRestart` 进行粗粒度区间重启 (相当于 `MidiLoop` 演示) 。需要精确度与相位回调时请使用 `InnerLoop`。

```
#if FEATURE_USE_MPTK && MPTK_PRO
using jp.kshoji.unity.midi.mptk;
using MidiPlayerTK;
using UnityEngine;

public sealed class MyInnerLoopSetup : MonoBehaviour
{
    public MidiFilePlayer filePlayer;

    void Start()
    {
        var loop = gameObject.AddComponent<MptkInnerLoopController>();
        loop.Source = filePlayer;
        // Start → Resume ... → End (Max 次) ; Max=0 表示无限
        loop.SetLoop(start: 0, resume: 480 * 4, end: 480 * 16, max: 3);
        loop.OnLoopExit.AddListener(() => Debug.Log("chorus loop done"));
        filePlayer.MPTK_Play();
    }
}
#endif
```

MIDI 2.0 输入适配器 — `MptkMidi2ManagerInputAdapter`

将 MPTK 播放器的回调作为 **MIDI 2.0 虚拟输入** 注入到 `Midi2Manager` (默认 `deviceId` : `mptk2:internal`) 。

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi.mptk;

MptkUtility.CreateMidi2ManagerInputAdapter(
    filePlayer,
    streamPlayer,
    deviceId: MptkUtility.DefaultMidi2InputDeviceId);
#endif
```

MIDI 1.0 的 7 位值会被缩放为 16 位 / 32 位以适配 UMP 后再注入。

MPE 输出 — MptkMpeOutput

通过 MpeManager 向 MPTK 虚拟设备发送 MPE 音符。内部会调用 SetupMpeZone，成员通道分配由 MpeManager 负责。

```
#if FEATURE_USE_MPTK
var mpe = gameObject.AddComponent<MptkMpeOutput>();
mpe.ConfigureZone();
mpe.SendNoteOn(60, 100);
#endif
```

Visual Scripting 节点（FEATURE_USE_MPTK + FEATURE_USE_VISUALSCRIPTING）

已在程序集 jp.kshoji.midi.mptk.visualscripting 中添加 MPTK 专用节点。

节点	类别	说明
MPTK Ensure Ready	MIDI/MPTK	MptkBootstrap.EnsureReady()
MPTK Panic	MIDI/MPTK	All Notes Off
MPTK Send Routed Note On	MIDI/MPTK	通过 MidiOutputRoutingPreset 发送 Note On
MPTK Configure MPE Zone	MIDI/MPTK	MptkMpeOutput.ConfigureZone()
MPTK Pipeline Configure	MIDI/MPTK	Pipeline 的 arp / drop patch / tempo
MPTK Writer Play	MIDI/MPTK	MptkWriterBridge.Play()
MPTK External Play	MIDI/MPTK	MptkExternalPlayback.Play(uri)
MPTK InnerLoop Apply	MIDI/MPTK	MptkInnerLoopController.SetLoop
MPTK List Play	MIDI/MPTK	MptkListPlayerBridge.Play / PlayAtIndex
MPTK SoundFont Load	MIDI/MPTK	MptkSoundFontLoader.Load
MPTK Spatializer Play	MIDI/MPTK	MptkSpatializerHost.Play
MPTK Effects Apply	MIDI/MPTK	SoundFont filter / reverb / chorus
MPTK Voice Lifecycle	MIDI/MPTK	暂停 / 恢复语音
MPTK Distance Audio Apply	MIDI/MPTK	距离 / Orientation
MPTK Chord Progression	MIDI/MPTK	播放 / 停止进行预设
MPTK Global Settings	MIDI/MPTK	SetRunInBackground

初次设置：

- 1. 启用 FEATURE_USE_MPTK 与 FEATURE_USE_VISUALSCRIPTING
- 2. 执行 Window > MIDI > Visual Scripting > Register MPTK Nodes
- 3. 在 Script Graph 中使用 MIDI/MPTK 类别节点

文档示例

示例	路径
Clock 同步	Assets/MIDI/Samples/DocumentationExamples/MptkClockSyncExample.cs
MIDI 2.0 输入	Assets/MIDI/Samples/DocumentationExamples/MptkMidi2InputAdapterExample.cs
MPE 输出	Assets/MIDI/Samples/DocumentationExamples/MptkMpeOutputExample.cs
Event Pipeline	Assets/MIDI/Samples/DocumentationExamples/MptkMidiEventPipelineExample.cs
Writer / External	Assets/MIDI/Samples/DocumentationExamples/MptkWriterExternalExample.cs
InnerLoop	Assets/MIDI/Samples/DocumentationExamples/MptkInnerLoopExample.cs
List / SF / Delayed / Channels	MptkListPlayerExample.cs / MptkSoundFontLoaderExample.cs / MptkDelayedNoteDispatcherExample.cs / MptkFilePlayerChannelsExample.cs
Spatializer	Assets/MIDI/Samples/DocumentationExamples/MptkSpatializerExample.cs
Timeline Marker	Assets/MIDI/Samples/DocumentationExamples/MptkTimelineMarkerExample.cs

以下示例位于 `Assets/MIDI/Samples/DocumentationExamples/` 目录下：

- **MIDI 1.0 → MPTK (虚拟输出 Sink)**

`Assets/MIDI/Samples/DocumentationExamples/MptkVirtualOutputSinkExample.cs`

- **MPTK → MIDI 1.0 (作为虚拟输入注入到 MidiManager)**

`Assets/MIDI/Samples/DocumentationExamples/MptkToMidiManagerInputExample.cs`

- **Bootstrap + SmfPlayer + 校验**

`Assets/MIDI/Samples/DocumentationExamples/MptkBootstrapExample.cs`

- **Clock 同步**

`Assets/MIDI/Samples/DocumentationExamples/MptkClockSyncExample.cs`

- **合成前事件管线**

`Assets/MIDI/Samples/DocumentationExamples/MptkMidiEventPipelineExample.cs`

- **Writer / External 往返**

`Assets/MIDI/Samples/DocumentationExamples/MptkWriterExternalExample.cs`

- **InnerLoop (循环区间)**

`Assets/MIDI/Samples/DocumentationExamples/MptkInnerLoopExample.cs`

- **List / SoundFont / Delayed / Channels**

`Assets/MIDI/Samples/DocumentationExamples/MptkListPlayerExample.cs` 等

- **Spatializer**

`Assets/MIDI/Samples/DocumentationExamples/MptkSpatializerExample.cs`

- **Timeline Marker Seek / InnerLoop**

`Assets/MIDI/Samples/DocumentationExamples/MptkTimelineMarkerExample.cs`

- **MIDI 2.0 输入适配器**

`Assets/MIDI/Samples/DocumentationExamples/MptkMidi2InputAdapterExample.cs`

- **MPE 输出**

`Assets/MIDI/Samples/DocumentationExamples/MptkMpeOutputExample.cs`

示例场景（已集成）

内置的示例场景也包含了可选的 MPTK 集成：

- MIDI 1.0 示例场景脚本：

`Assets/MIDI/Samples/Scripts/MidiSampleScene.cs`

（添加了一个开关，用于使用基于 MPTK 的虚拟输出设备）

- MIDI 2.0 示例场景脚本：

`Assets/MIDI/Samples/Scripts/Midi2SampleScene.cs`

（添加了一个开关，用于将 MIDI 2.0 发送的内容镜像到 MPTK MIDI 1.0 虚拟 Sink 中）

ID 选择、组与路由

推荐规范：

- 使用带有明显虚拟前缀的名称（例如 `mptk:internal` , `virtual:sequencer` , `test:device` ）。
- 除非您有意模拟多个组，否则请使用 `group = 0` 。

由于虚拟设备会出现在设备集合中，您可以构建 UI 让用户像选择硬件设备一样选择它。

故障排除

“在编辑器中可以编译，但在 CI 或其他机器上失败”

- 确保仅在项目中确实存在 MPTK 资源时才启用 `FEATURE_USE_MPTK`。

“未接收到事件”

- 确认虚拟设备已注册为**输入 (Input)**（这样注入的事件才会被视为来自输入设备）。
- 确认您的处理器已在 `MidiManager` 中注册。
- 确认 `deviceId` 和 `group` 与您预期的匹配。

“MPTK Sink 没有声音”

- 确认虚拟设备已注册为**输出 (Output)**。
- 确认 Sink 的 `deviceId` 与您发送消息的目标设备一致。
- 确认项目中的 MPTK 全局设置/资源（`SoundFont` / 配置）是有效的。
- 使用 `MptkUtility.ValidateSetup()` 或 `MptkBootstrap.Validate()` 检查 `SoundFont` / `AudioSource` / 虚拟设备注册。

“SmfPlayer 到 MPTK 没有声音”

- 在同一 `GameObject`（或子对象）上添加 `MptkSmfPlayerOutput` 并调用 `Configure()`。
- 确认 `SmfPlayer.outputDeviceId` 为 `mptk:internal`（或 Bootstrap 设置的 ID）。
- 进入 Play 模式前必须已完成 `MptkBootstrap.EnsureReady()`。

“Scriptable Audio UMP 到 MPTK 没有声音”

- 确认 `FEATURE_SCRIPTABLE_AUDIO` 与 `FEATURE_USE_MPTK` 均已启用。
- 添加 `MptkDspUmpSequenceOutput`，并确认 `MidiDspUmpMidi2OutBridge` 已启用（Bootstrap 的 UMP Out 或手动接线）。
- 确认已调用 `MidiManager.InitializeMidi2()`。

“Panic 后仍有残留声音”

- 直接调用 `MptkBootstrap.Panic()` 或 `MptkMidiDevice.PanicAll()`。
- 若同时向硬件和 MPTK 发送，则需要在各自的输出目标分别执行 Panic。

相关文档

- [MIDI 1.0 \(MidiManager\)](#)
- [Unity 生态系统集成 — UMP 序列](#)
- [示例项目](#)